

C Struct Exercises: Points and Rectangles

Based on ECM2433 CA0 and L07 struct material. Practises defining structs with typedef, initialising and accessing fields, writing functions that take struct parameters, pass-by-value vs pass-by-pointer, and modelling geometric types.

What You Need To Know

1. Quick recap: typedef struct

The typedef form lets you use the short name directly without writing struct each time.

```
typedef struct {
    double x;
    double y;
} Point;

Point p;           /* declares a Point without writing 'struct Point' */
```

2. Quick recap: field access

Initialise with a brace list in field order. Access fields with the dot operator on a struct value, or the arrow operator on a pointer to a struct.

```
Point origin = {0.0, 0.0};
Point p      = {3.0, 4.0};

printf("x = %f\n", p.x); /* dot: value */

Point *ptr = &p;
printf("x = %f\n", ptr->x); /* arrow: pointer */
```

3. Passing structs to functions

Passing a struct by value copies all its fields into the function's local parameter. Any modification inside the function does not affect the caller's copy. Pass a pointer if the function needs to modify the struct.

```
/* by value: caller's copy is unchanged */
double distance_sq(Point a, Point b) {
    double dx = a.x - b.x;
    double dy = a.y - b.y;
    return dx*dx + dy*dy;
}

/* by pointer: modifies the caller's struct */
void move_point(Point *p, double dx, double dy) {
    p->x += dx;
    p->y += dy;
}
```

4. Structs containing other structs: Rectangle

A struct field can itself be a struct. A rectangle with sides parallel to the axes is fully described by its bottom-left and top-right corners.

```
typedef struct {
    Point bottom_left;
    Point top_right;
} Rectangle;

Rectangle r = {{1.0, 2.0}, {5.0, 6.0}};

/* access nested fields */
printf("%f\n", r.bottom_left.x); /* 1.0 */
printf("%f\n", r.top_right.y);   /* 6.0 */
```

5. Writing functions on geometric structs

The width of a rectangle is the difference in x coordinates of its corners; the height is the difference in y coordinates. A point is inside the rectangle if its x coordinate is between the two corners' x values and similarly for y.

```
double area(Rectangle r) {
    double width = r.top_right.x - r.bottom_left.x;
    double height = r.top_right.y - r.bottom_left.y;
    return width * height;
}

/* returns 1 if p is strictly inside r, 0 otherwise */
```

```

int inside(Point p, Rectangle r) {
    return p.x > r.bottom_left.x && p.x < r.top_right.x
        && p.y > r.bottom_left.y && p.y < r.top_right.y;
}

```

Practice Questions

- 1 Repair this broken Point definition and explain what the final Point name is doing: `typedef struct { double x double y; } Point;`
- 2 Given `Point path[3] = {{0,0},{3,4},{6,8}};` and `Point *p = &path[1];`, write one dot expression and one arrow expression that both read the middle point's y coordinate.

Question 3 — state what the following prints:

```

typedef struct { double x; double y; } Point;
Point a = {1.5, 2.5};
Point b = {4.5, 0.5};
printf("%.1f\n", a.x + b.y);

```

- 3 State what the code above prints.
- 4 Write `double distance_sq(Point a, Point b)` that returns the squared distance between the two points. (No need for `sqrt`.)
- 5 Write a `typedef struct` for a `Rectangle` defined by a `bottom_left` `Point` and a `top_right` `Point`.
- 6 Write `double area(Rectangle r)` that returns the area of the rectangle.
- 7 Write `int inside(Point p, Rectangle r)` that returns 1 if `p` is strictly inside `r` (not on the boundary), 0 otherwise.

Question 8 — the function below is meant to translate a point by `(dx, dy)` but the change is not visible in the caller:

```

void move_point(Point p, double dx, double dy) {
    p.x += dx;
    p.y += dy;
}

int main() {
    Point pt = {1.0, 2.0};
    move_point(pt, 1.0, 1.0);
    printf("%.1f %.1f\n", pt.x, pt.y); /* still prints 1.0 2.0 */
}

```

- 8 Explain why the change does not appear in the caller and fix the function so it does. Show the corrected call site as well.

Question 9 — choose the better parameter style:

```

double distance_sq(Point a, Point b) {
    double dx = a.x - b.x;
    double dy = a.y - b.y;
    return dx*dx + dy*dy;
}

```

- 9 For this function, explain why passing each `Point` by value is reasonable, rather than passing pointers.
- 10 What does `sizeof(Point)` return for the `Point` defined in question 1, and why might the answer be larger than `2 * sizeof(double)`?

Mark Scheme

Attempt all questions before checking. Any correct implementation scores full marks.

- 1 `typedef struct { double x; double y; } Point;`. The final `Point` is the typedef alias, so you can declare `Point p;` without writing `struct`.
- 2 Dot form: `path[1].y`. Arrow form: `p->y`.
- 3 `2.0` — `a.x` is 1.5, `b.y` is 0.5, sum is 2.0.
- 4 `double dx = a.x - b.x; double dy = a.y - b.y; return dx*dx + dy*dy;`
- 5 `typedef struct { Point bottom_left; Point top_right; } Rectangle;`
- 6 `return (r.top_right.x - r.bottom_left.x) * (r.top_right.y - r.bottom_left.y);`
- 7 `return p.x > r.bottom_left.x && p.x < r.top_right.x && p.y > r.bottom_left.y && p.y < r.top_right.y;`
- 8 Structs are passed by value in C — the function receives its own copy of the `Point`, so changes to that copy are discarded when the function returns. Fix: change the parameter to `Point *p` and use `p->x += dx; p->y += dy;`. Call site: `move_point(&pt, 1.0, 1.0);`
- 9 The function only reads the two points and does not need to modify the caller's variables. `Point` is also small here, so copying two doubles per argument is simple and reasonable. Passing by value also prevents accidental writes to the caller's data.
- 10 `sizeof(Point)` is typically 16 bytes (two double fields at 8 bytes each). On some platforms it could be larger due to struct padding/alignment, but for two doubles of the same type, padding is unlikely here.

Useful Code Patterns

```
#include <stdio.h>
#include <stdbool.h>

typedef struct { double x; double y; } Point;
typedef struct { Point bottom_left; Point top_right; } Rectangle;

double area(Rectangle r) {
    return (r.top_right.x - r.bottom_left.x)
        * (r.top_right.y - r.bottom_left.y);
}

int inside(Point p, Rectangle r) {
    return p.x > r.bottom_left.x && p.x < r.top_right.x
        && p.y > r.bottom_left.y && p.y < r.top_right.y;
}

void move_point(Point *p, double dx, double dy) {
    p->x += dx;
    p->y += dy;
}
```